



Exercício 3.3: Consertar o manifesto quebrado

Aluno: Luiz Brito • **Data:** 18/09/2026 • **Disciplina:** Cloud Native Development (CP1)

Ambiente

- Cluster local **kind** v0.32.0, Kubernetes **v1.36.1**, contexto **kind-fiap-store**, namespace **store-dev**.
- Manifesto original salvo como **01-broken.yaml** (numeração de linhas abaixo refere-se a este arquivo).
- Cada correção foi aplicada numa cópia de trabalho (**work.yaml**) para mostrar a progressão. O resultado final é o **02-fixed.yaml**. Nas saídas de erro o caminho completo da cópia foi abreviado para **work.yaml**.
- Pod cliente **curl-client** (imagem **curlimages/curl:8.10.1**) para os testes de conectividade.

Manifesto quebrado

```
apiVersion: v1
kind: Pod
metadata:
  name: store-api
  namespace: store-dev
  labels:
    app: store-api
    tier: backend
    env: dev
spec:
  containers:
    - name: store-api
      image: store-api:1.0
      ports:
        - name: http
          containerPort: 8000
      livenessprobe:
        httpGet:
          path: /health
          port: 8000
---
apiVersion: extensions/v1beta1
kind: Service
metadata:
  name: store-api
  namespace: store-dev
spec:
  type: ClusterIP
  selector:
    app: store-api
    env: prod
  ports:
    - port: 80
      targetPort: 9000
```

Tabela dos 5 defeitos

#	Linha	Tipo	Sintoma	Comando revelador	Correção
i	8	Sintaxe YAML	yaml: line 8: mapping values are not allowed in this context	kubectl apply --dry-run=server -f	Alinhar <code>tier: backend</code> com <code>app</code> e <code>env</code> (4 espaços)
ii	17	API inválida (campo desconhecido)	strict decoding error: unknown field "spec.containers[0].livenessprobe"	kubectl apply --dry-run=server -f	<code>livenessprobe</code> para <code>livenessProbe</code>

#	Linha	Tipo	Sintoma	Comando revelador	Correção
iii	22	API inválida (versão inexistente)	<code>no matches for kind "Service" in version "extensions/v1beta1"</code>	<code>kubectl apply --dry-run=server -f</code>	<code>apiVersion: v1</code>
iv	31	Valor incoerente com outro objeto	Sem erro no <code>apply</code> . <code>Endpoints</code> fica <code><none></code>	<code>kubectl get endpoints store-api -n store-dev</code>	<code>env: prod</code> para <code>env: dev</code> (label do Pod)
v	34	Valor incoerente com a aplicação	Sem erro no <code>apply</code> . <code>Endpoints</code> existe, mas o <code>curl</code> falha (<code>exit 7</code>)	<code>kubectl exec curl-client - curl -sS http://store-api/health</code>	<code>targetPort: 9000</code> para <code>targetPort: 8000</code>

Progressão das mensagens de erro

Passo 0: manifesto original

```
$ kubectl apply --dry-run=server -f 01-broken.yaml
error: error parsing 01-broken.yaml: error converting YAML to JSON: yaml: line 8: mapping values are not allowed in this context
```

O primeiro filtro é o **parser YAML**: o arquivo nem vira objeto. Corrijo o defeito (i).

Passo 1: indentação corrigida

```
$ kubectl apply --dry-run=server -f work.yaml
resource mapping not found for name: "store-api" namespace: "store-dev" from "work.yaml": no matches for kind "Service" in version "extensions/v1beta1"
ensure CRDs are installed first
Error from server (BadRequest): error when creating "work.yaml": Pod in version "v1" cannot be handled as a Pod: strict decoding error: unknown field "spec.containers[0].livenessprobe"
```

O YAML agora é válido e aparecem dois erros de uma vez, de camadas diferentes: o kubectl não consegue mapear `Service` em `extensions/v1beta1` (descoberta da API) e o servidor rejeita o campo desconhecido do Pod (**esquema estrito**). Corrijo o defeito (ii).

Passo 2: `livenessProbe` corrigido

```
$ kubectl apply --dry-run=server -f work.yaml
pod/store-api created (server dry run)
error: resource mapping not found for name: "store-api" namespace: "store-dev" from "work.yaml": no matches for
kind "Service" in version "extensions/v1beta1"
ensure CRDs are installed first
```

O Pod passou. Sobra o Service com a `apiVersion` que não existe mais. Corrijo o defeito (iii).

Passo 3: `apiVersion` corrigida

```
$ kubectl apply --dry-run=server -f work.yaml
pod/store-api created (server dry run)
service/store-api created (server dry run)
```

A validação estática (sintaxe, API, esquema, admission em dry-run) passa por completo. Restam os defeitos (iv) e (v).

Passo 4: `apply` real, com (iv) e (v) ainda presentes

```
$ kubectl apply -f work.yaml
pod/store-api created
service/store-api created
$ kubectl get pod,svc,endpoints -n store-dev
```

NAME	READY	STATUS	RESTARTS	AGE
pod/store-api	1/1	Running	0	1s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/store-api	ClusterIP	10.96.136.50	<none>	80/TCP	1s

NAME	ENDPOINTS	AGE
endpoints/store-api	<none>	1s

```
$ kubectl exec curl-client -n store-dev -- curl -sS -m 5 http://store-api/health
curl: (7) Failed to connect to store-api port 80 after 2 ms: Could not connect to server
command terminated with exit code 7
```

Tudo é criado sem nenhum erro, o Pod fica `1/1 Running`, mas o Service tem `ENDPOINTS <none>` e o `curl` falha.

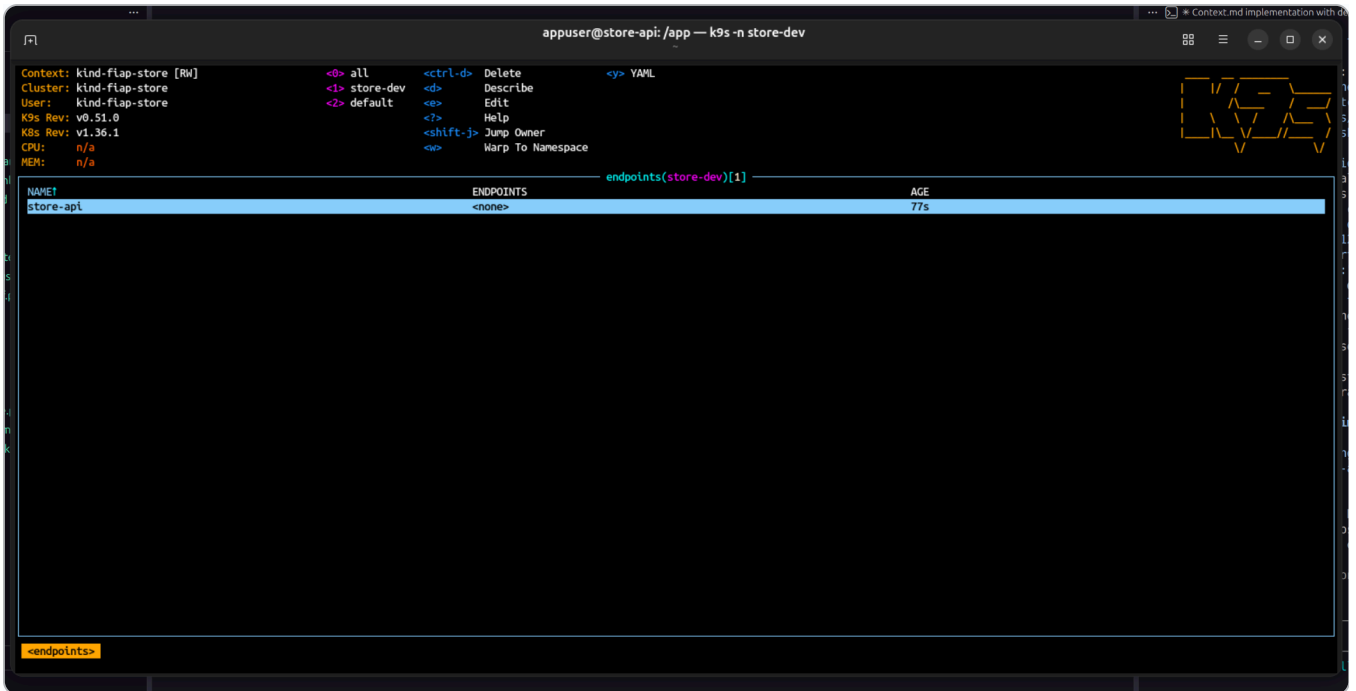


Figura 1 (k9s): `store-api` com `ENDPOINTS <none>`, o sintoma do defeito (iv).

Passo 5: seletor corrigido (iv)

```
$ kubectl apply -f work.yaml
pod/store-api unchanged
service/store-api configured
$ kubectl get endpoints store-api -n store-dev
NAME      ENDPOINTS      AGE
store-api  10.244.0.12:9000 12s
$ kubectl exec curl-client -n store-dev -- curl -sS -m 5 http://store-api/health
curl: (7) Failed to connect to store-api port 80 after 1 ms: Could not connect to server
command terminated with exit code 7
```

Agora o Service acha o Pod (`10.244.0.12`), mas encaminha para a porta 9000, onde nada escuta. A conexão falha logo (1 ms), não por timeout como a apostila descreve: a porta fechada recusa imediatamente. O comportamento é o mesmo: o erro só aparece em tempo de execução.

Passo 6: `targetPort` corrigido (v)

```
$ kubectl apply -f work.yaml
pod/store-api unchanged
service/store-api configured
$ kubectl get endpoints store-api -n store-dev
NAME      ENDPOINTS      AGE
store-api  10.244.0.12:8000 13s
$ kubectl exec curl-client -n store-dev -- curl -sS -m 5 http://store-api/health
{"status":"ok","host":"store-api"}
```

Diff completo (broken para fixed)

```
@@ -5,7 +5,7 @@
  namespace: store-dev
  labels:
    app: store-api
-   tier: backend
+   tier: backend
  env: dev
  spec:
    containers:
@@ -14,12 +14,12 @@
    ports:
      - name: http
        containerPort: 8000
-   livenessprobe:
+   livenessProbe:
      httpGet:
        path: /health
        port: 8000
---
- apiVersion: extensions/v1beta1
+ apiVersion: v1
  kind: Service
  metadata:
    name: store-api
@@ -28,7 +28,7 @@
  type: ClusterIP
  selector:
    app: store-api
-   env: prod
+   env: dev
  ports:
    - port: 80
-     targetPort: 9000
+     targetPort: 8000
```

Manifesto corrigido (02-fixed.yaml)

```
apiVersion: v1
kind: Pod
metadata:
  name: store-api
  namespace: store-dev
  labels:
    app: store-api
    tier: backend
    env: dev
spec:
  containers:
    - name: store-api
      image: store-api:1.0
      ports:
        - name: http
          containerPort: 8000
      livenessProbe:
        httpGet:
          path: /health
          port: 8000
---
apiVersion: v1
kind: Service
metadata:
  name: store-api
  namespace: store-dev
spec:
  type: ClusterIP
  selector:
    app: store-api
    env: dev
  ports:
    - port: 80
      targetPort: 8000
```

Estado final: Service com um endpoint e `curl` bem-sucedido a partir de um Pod cliente.

```
appuser@store-api: /app — k9s -n store-dev
Context: kind-fiap-store [RW]
Cluster: kind-fiap-store
User: kind-fiap-store
K9s Rev: v0.51.0
K8s Rev: v1.36.1
CPU: n/a
MEM: n/a

Describe(store-dev/store-api)

Name: store-api
Namespace: store-dev
Labels: endpoints.kubernetes.io/managed-by=endpoint-controller
Annotations: endpoints.kubernetes.io/last-change-trigger-time: 2026-09-19T02:06:14Z
Subsets:
  Addresses: 10.244.0.12
  NotReadyAddresses: <none>
  Ports:
    Name Port Protocol
    ---- --
    <unset> 8000 TCP
Events: <none>
```

Figura 2 (k9s, `describe` do Endpoints): `Addresses: 10.244.0.12`, `NotReadyAddresses: <none>` e porta `8000/TCP`.

```
View Go Run Terminal Help
<K9s-Shell>> Pod: store-dev/curl-client | Container: curl-client
~ $ curl -s http://store-api/health
{"status":"ok","host":"store-api"}~ $
```

Figura 3 (k9s, shell no `curl-client`): `curl -s http://store-api/health` devolve `{"status":"ok","host":"store-api"}`.

Pergunta de fechamento

Quais dois defeitos não geram erro no `kubectl apply` e o que isso ensina?

Os defeitos (iv) e (v). No (iv), o seletor `env: prod` não bate com o label `env: dev` do Pod: o Service é criado válido, só que com `Endpoints` vazio. No (v), o `targetPort: 9000` é um número válido, mas nada escuta nessa porta no container: o objeto é aceito e a falha só aparece quando o tráfego chega. O `apply` só valida o formato de cada objeto isoladamente (sintaxe, esquema e admission). Ele não verifica se um objeto combina com outro (seletor e labels) nem se o valor combina com a aplicação (porta real). Um `apply` bem-sucedido significa "objeto aceito", não

”sistema funcionando”, então é preciso conferir com `kubectl get endpoints` e com um `curl` de dentro do cluster.

Conclusão

Os cinco defeitos foram identificados, classificados e corrigidos em ordem. Os três primeiros são pegos pelo `--dry-run=server` (parser YAML, descoberta da API e esquema estrito). Os dois últimos passam pelo `apply` e só aparecem em `Endpoints` e na conectividade. O `02-fixed.yaml` final entrega um Service com um endpoint e `curl` bem-sucedido.